UNIT - IV

5 Database Transaction Management

Syllabus

Introduction to Database Transaction, Transaction states, ACID properties, Concept of Schedule, Serial Schedule. ***Serializability :*** *Conflict and View, Cascaded Aborts, Recoverable and Non-recoverable Schedules.* ***Concurrency Control :*** *Lock-based, Time-stamp based Deadlock handling.* ***Recovery methods :*** *Shadow-Paging and Log-Based Recovery, Checkpoints.* ***Log-Based Recovery :*** *Deferred Database Modifications and Immediate Database Modifications.*

Contents

Part I : Transaction Management

5.1 Introduction to Database Transaction

- **Definition of Transaction :** A transaction can be defined as a **group of tasks** that form a single logical unit. For example - Suppose we want to withdraw ₹ 100 from an account then we will follow following operations :
  1) Check account balance
  2) If sufficient balance is present request for withdrawal.
  3) Get the money
  4) Calculate Balance = Balance – 100
  5) Update account with new balance.

The above mentioned **five steps** denote **one transaction**.

- A database is a collection of named data items.
- **Granularity or size of data items :** The size of data items can be a field, a record or a whole disk block.
- Basic operations on data item A are -

1. read_item(X)	2. write_item(X)

1. **read_item(X) :** This is a reading operation in which database item named X is read into a programming variable. We can name the program variable as X for simplification.
2. **write_item(X) :** This operation writes the value of program variable X into the database item which is also named as X.

- Basic unit of data transfer from the disk to the computer main memory is one block.
- **read_item(X) command includes the following steps :**

  Step 1 : Find the address of the disk block that contains item X.

  Step 2 : Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).

  Step 3 : Copy item X from the buffer to the program variable named X.

- **write_item(X) command includes the following steps :**

  Step 1 : Find the address of the disk block that contains item X.

  Step 2 : Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).

  Step 3 : Copy item X from the program variable named X into its correct location in the buffer.

  Step 4 : Store the updated block from the buffer back to disk.

- **Example of sample transaction performing read and write operations**

```
read_item(X);
X=X+M;
Write_item(X);
```

- **Transaction Notations :**

The transaction notations focuses on read and write operations. For example – Following are two transactions denoted by T1 and T2

```
T1:b1;r1(X);w1(X);r1(Y);W1(Y);e1;
T2:b2;r2(X);r2(Y);e2
```

The r and w represents the read and write operations. The b1 and b2 represents the beginning and e1 and e2 represents ending of transaction.

5.2 Transaction States

SPPU : Dec..-17, Marks 8

Each transaction has following five states :

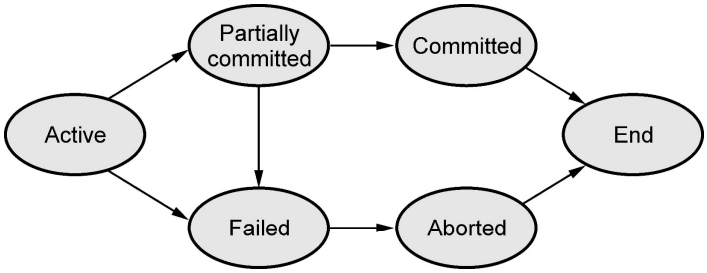


Fig. 5.2.1 Transaction states

1) **Active :** This is the first state of transaction. For example : Insertion, deletion or updation of record is done here. But data is not saved to database.
2) **Partially committed :** When a transaction executes its final operation, it is said to be in a partially committed state.
3) **Failed :** A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.
4) **Aborted :** If a transaction is failed to execute, then the database recovery system will make sure that the database is in its previous consistent state. If not, it brings the database to consistent state by aborting or rolling back the transaction.
5) **Committed :** If a transaction executes all its operations successfully, it is said to be committed. This is the last step of a transaction, if it executes without fail.

Example 5.2.1 *Define a transaction. Then discuss the following with relevant examples :*

1. A read only transaction 2. A read write transaction 3. An aborted transaction

Solution :

1) Read only transaction

T1
Read(A)
Read(B)
Display(A – B)

2) A read write transaction

T1
Read(A)
A=A+100
Write(A)

3)

T1	T2	Description
Read(A)		Assume A=100
A=A+50		A=150
Write(A)		
	Read(A)	A=150
	A=A+100	A=250
RollBack		A=100 (restore back to original value which is before Transaction T1)
	Write(A)	

Review Question

1. *Transaction during its execution should be in one of the different states at any point of time, explain the different states of transactions during its execution.*

SPPU : Dec 17, End Sem, Marks 8

5.3 ACID Properties

SPPU : Dec.-18,19, May-18,19, Marks 8

1) Atomicity :

- This property states that each transaction must be considered as a **single unit and must be completed fully or not completed at all.**
- No transaction in the database is left half completed.
- Database should be in a state either before the transaction execution or after the transaction execution. It **should not be in a state 'executing'.**
- For example - In above mentioned withdrawal of money transaction all the five steps must be completed fully or none of the step is completed. Suppose if transaction gets failed after step 3, then the customer will get the money but the balance will not be updated accordingly. The state of database should be either at before ATM withdrawal (i.e customer without withdrawn money) or after ATM withdrawal (i.e. customer with money and account updated). This will make the system in consistent state.

2) Consistency :

- The database must remain in consistent state after performing any transaction.
- **For example :** In ATM withdrawal operation, the balance must be updated appropriately after performing transaction. Thus the database can be in consistent state.

3) Isolation :

- In a database system where **more than one transaction** are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system.
- No transaction will affect the existence of any other transaction.
- **For example :** If a bank manager is checking the account balance of particular customer, then manager should see the balance either before withdrawing the money or after withdrawing the money. This will make sure that each individual transaction is completed and any other dependent transaction will get the consistent data out of it. Any failure to any transaction will not affect other transaction in this case. Hence it makes all the transactions consistent.

4) Durability :

- The database should be **strong enough** to handle any **system failure.**
- If there is any set of insert /update, then it should be able to handle and commit to the database.
- If there is any **failure,** the database should be able to **recover** it to the consistent state.
- For example : In ATM withdrawal example, if the system failure happens after customer getting the money then the system should be strong enough to update Database with his new balance, after system recovers. For that purpose the system has to keep the **log of each transaction and its failure.** So when the system recovers, it should be able to know when a system has failed and if there is any pending transaction, then it should be updated to Database.

Review Question

1. *State and explain in brief the ACID properties. During execution of transaction, a transaction passes through several states, until it finally commits or aborts. List all possible sequences of states through which a transaction may pass. Explain why each state transition occurs.*

SPPU : May-18,19, Dec.-18,19, End Sem, Marks 8

5.4 Concept of Schedule

Schedule is an order of multiple transactions executing in concurrent environment. Following Fig. 5.4.1 represents the types of schedules.

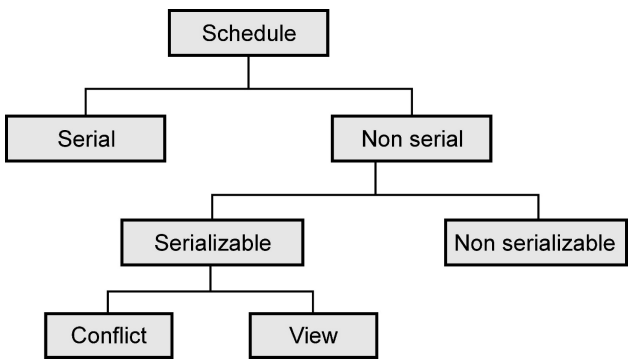


Fig. 5.4.1 : Types of schedule

Serial schedule : The schedule in which the transactions execute one after the other is called **serial schedule**. It is consistent in nature. For example : Consider following two transactions T1 and T2.

T1	T2
R(A)	
W(A)	
R(B)	
W(B)	
	R(A)
	W(A)
	R(B)
	W(B)

All the operations of transaction T1 on data items A and then B executes and then in transaction T2 all the operations on data items A and B execute. The **R stands for read** operation and **W stands for write** operation.

Non serial schedule : The schedule in which operations present within the transaction are intermixed. This may lead to conflicts in the result or inconsistency in the resultant data.

For example -

Consider following two transactions,

T1	T2
R(A)	
W(A)	
	R(A)
	W(B)
R(A)	
W(B)	
	R(B)
	W(B)

The above transaction is said to be non serial which result in inconsistency or conflicts in the data.

5.5 Serializability : Conflict and View

SPPU : Dec.-17,18,19, May-18, Marks 9

- When multiple transactions run concurrently, then it may lead to inconsistency of data (i.e. change in the resultant value of data from different transactions).

- Serializability is a concept that helps to identify which non serial schedule and find the transaction equivalent to serial schedule.

T1	A	B	T2
Initial Value	100	100	
A=A – 10			
W(A)			
B=B+10			
W(B)			
	90	110	
			A=A–10
			W(A)
	80	110	

- In above transactions initially T1 will read the values from database as A = 100, B = 100 and modify the values of A and B. But transaction T2 will read the modified value i.e. 90 and will modify it to 80 and perform write operation. Thus at the end of transaction T1 value of A will be 90 but at end of transaction T2 value of A will be 80. Thus conflicts or inconsistency occurs here. This sequence can be converted to a sequence which may give us consistent result. This process is called **serializability**.

Difference between serial schedule and serializable schedule

Sr. No.	**Serial** schedule	Serializable **schedule**
1	**No concurrency** is allowed in serial schedule.	**Concurrency** is allowed in serializable schedule.
2	In serial schedule, if there are two transactions executing at the same time and no interleaving of operations is permitted, then following can be the possibilities of execution - i) Execute all the operations of transactions T_1 in a sequence and then execute all the operations of transactions T_2 in a sequence. ii) Execute all the operations of transactions T_2 in a sequence and then execute all the operations of transactions T_1 in a sequence.	In serializable schedule, if there are two transactions executing at the same time and interleaving of operations is allowed there can be different possible orders of executing an individual operation of the transactions.

Example of serial schedule

T_1	T_2
Read(A)	
A=A–50	
Write(A)	
Read(B)	
B=B+100	
Write(B)	
	Read(A)
	A=A+10
	Write(A)

Example of serializable schedule

T_1	T_2
Read(A)	
A=A–50	
Write(A)	
	Read(B)
	B=B+100
	Write(B)
Read(B)	
Write(B)	

- **There are two types of serializabilities :** Conflict serializability and view serializability.

5.5.1 Conflict Serializability

Definition : Suppose T_1 and T_2 are two transactions and I_1 and I_2 are the instructions in T_1 and T_2 respectively. Then these two transactions are said to be conflict serializable, if both the instruction access the data item d, and at least one of the instruction is write operation.

What is conflict ? : In the definition **three conditions** are specified for a conflict in conflict serializability -

1) There should be **different transactions**
2) The operations must be performed on **same data** items
3) **One of the operation** must be the **Write (W)** operation.

- We can test a given schedule for conflict serializability by constructing a **precedence graph** for the schedule, and by searching for absence of cycles in the graph.
- Predence graph is a directed graph, consisting of G = (V,E) where V is set of vertices and E is set of edges. The set of vertices consists of all the transactions participating in the schedule. The set of edges consists of all edges $T_i \rightarrow T_j$ for which one of three conditions holds :
  1. T_i executes write(Q) before T_j executes read(Q).

2. T_i executes read(Q) before T_j executes write(Q).
3. T_i executes write(Q) before T_j executes write(Q).

- A serializability order of the transactions can be obtained by finding a linear order consistent with the partial order of the precedence graph. This process is called **topological sorting**.

Testing for serializability

- Following method is used for testing the serializability : To test the conflict serializability we can draw a graph G = (V, E) where V = Vertices which represent the number of transactions.

E = Edges for conflicting pairs.

Step 1 : Create a node for each transaction.

Step 2 : Find the conflicting pairs (RW, WR, WW) on the same variable (or data item) by different transactions.

Step 3 : Draw edge for the given schedule. Consider following cases

1. T_i executes write(Q) before T_j executes read(Q), then draw edge from T_i to T_j.
2. T_i executes read(Q) before T_j executes write(Q) , then draw edge from T_i to T_j.
3. T_i executes write(Q) before T_j executes write(Q), , then draw edge from T_i to T_j.

Step 4 : Now, if precedence graph is cyclic then it is a non conflict serializable schedule and if the precedence graph is acyclic then it is conflict serializable schedule.

Example 5.5.1 *Consider the following two transactions and schedule (time goes from top to bottom). Is this schedule conflict-serializable ? Explain why or why not.*

T_1	T_2
R(A)	
W(A)	
	R(A)
	R(B)
R(B)	
W(B)	

Solution :

Step 1 : To check whether the schedule is conflict serializable or not we will check from top to bottom. Thus we will start reading from top to bottom as,

T_1 : R(A) -> T_1 : W(A) ->T_2 : R(A) -> T_2 : R(B) ->T_1 : R(B) -> T_1 : W(B)

Step 2 : We will find **conflicting operations**. Two operations are called as **conflicting operations** if all the following conditions hold true for them -

i) Both the **operations belong to different transactions**.

ii) Both the **operations are on same data item**.

iii) **At least one** of the two operations is a **write operation**.

From above given example in the top to bottom scanning we find the conflict as T_1 : W(A) ->T_2 : R(A).

i) Here note that there are two different transactions T_1 and T_2,

ii) Both work on same data item i.e. A and

iii) One of the operation is write operation.

Step 3 : We will build a precedence graph by drawing one node from each transaction. In above given scenario as there are two transactions, there will be two nodes namely T_1 and T_2.

Fig. 5.5.1

Step 4 : Draw the edge between conflicting transactions. For example in above given scenario, the conflict occurs while moving from T_1 : W(A) to T_2 : R(A). Hence edge must be from T_1 and T_2.

Fig. 5.5.2

Step 5 : Repeat the step 4 while reading from top to bottom. Finally the precedence graph will be as follows

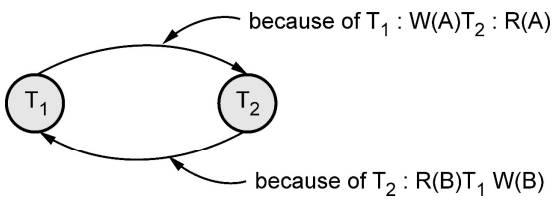


Fig. 5.5.3 : Precedence graph

Step 6 : Check if any cycle exists in the graph. Cycle is a path using which we can start from one node and reach to the same node. If the is cycle found then schedule is not conflict serializable. In the step 5 we get a graph with cycle, that means given schedule is not conflict serializable.

Example 5.5.2 *Check whether following schedule is conflict serializable or not. If it is not conflict serializable then find the serializability order.*

T1	T2	T3
R(A)		
	R(B)	
		R(B)
	W(B)	
W(A)		
		W(A)
	R(A)	
	W(A)	

Solution :

Step 1 : We will read from top to bottom, and build a precedence graph for conflicting entries. We will build a precedence graph by drawing one node from each transaction. In above given scenario as there are three transactions, there will be two nodes namely T1 T2, and T3

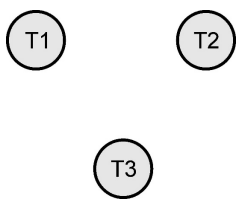
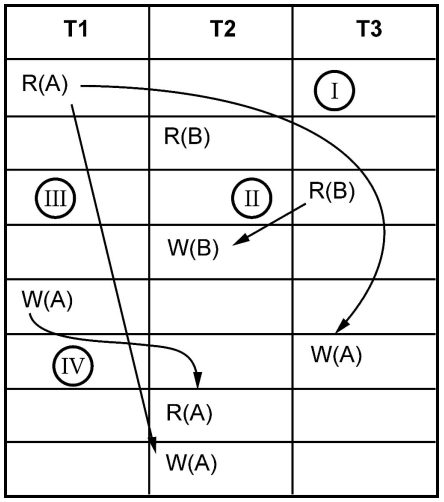
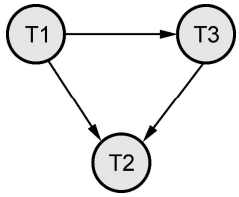


Fig. 5.5.4

Step 2 : The conflicts are found as follows –



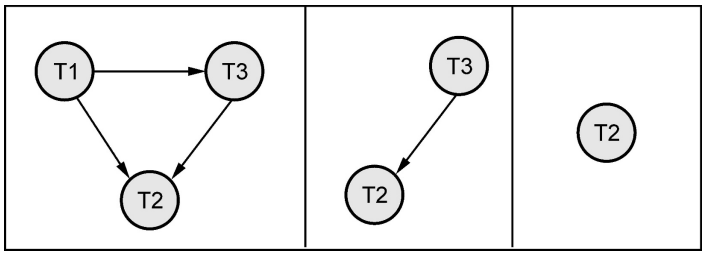
Step 3 : The precedence graph will be as follows –



Step 4 : As there is **no cycle** in the precedence graph, the given sequence is **conflict serializable**. Hence we can convert this non serial schedule to serial schedule. For that purpose we will follow these steps to find the serializable order.

Step 5 : A **serializability order** of the transactions can be obtained by finding a linear order consistent with the partial order of the precedence graph. This process is called topological sorting.

Step 6 : Find the vertex which has no incoming edge which is T1. If we delete T1 node then T3 is a node that has no incoming edge. If we delete T3, then T2 is a node that has no incoming edge.



Thus the nodes can be deleted in a order T1, T3 and T2. Hence the order will be T1-T3-T2

Example 5.5.3 *Check whether the below schedule is conflict serializable or not.*

{B2,r2(X),b1,r1(X),W1(X),r1(Y),W1(Y),W2(X),e1,C1,e2,C2}

Solution : b2 and b1 represents **begin transaction 2** and **begin transaction 1**. Similarly, e1 and e2 represents **end transaction 1** and **end transaction 2**.

We will rewrite the schedule as follows -

T_1	T_2
	r2(X)
r1(X)	
W1(X)	
r1(Y)	
W1(Y)	
	W2(X)

Step 1 : We will find conflicting operations. Two operations are called as conflicting operations if all the following conditions hold true for them -

i) Both the **operations belong to different transactions**.

ii) Both the operations are on **same data item**.

iii) **At least one** of the two operations is **a write operation**

The conflicting entries are as follows -

T_1	T_2
	$R_2(X)$
$R_1(X)$	
$W_1(X)$	
$R_1(Y)$	
$W_1(Y)$	
	$W_2(X)$

Step 2 : Now we build a precedence graph for conflicting entries.

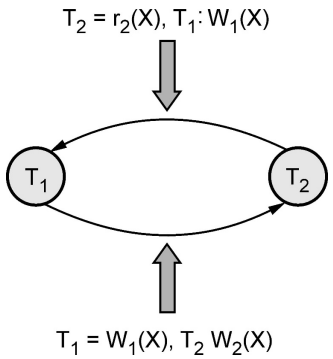


Fig. 5.5.5

As there are two transactions only two nodes are present in the graph.

Step 3 : We get a graph with cycle, that means given schedule is **not conflict serializable**.

Example 5.5.4 *Consider the three transactions T1, T2, and T3 and schedules S1 and S2 given below. Determine whether each schedule is serializable or not ? If a schedule is serializable write down the equivalent serial schedule(S).*

T1: R1(x) R1(z);W1(x);

T2: R2(x);R2(y);W2(z);W2(y)

T3:R3(x);R3(y);W3(y);

S1: R1(x);R2(z);R1(z);R3(x);R3(y);W1(x);W3(y);R2(y);W2(z);W2(y);

S2: R1(x);R2(z);R3(x);R1(z);R2(y);R3(y);W1(x);W2(z);W3(y);W2(y);

Solution :

Step 1 : We will represent the schedule S1 as follows

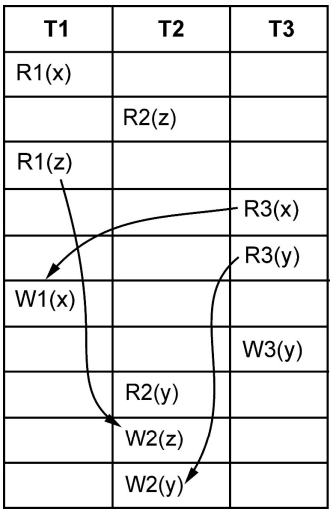
T1	T2	T3
R1(x)		
	R2(z)	
R1(z)		
		R3(x)
		R3(y)
W1(x)		
		W3(y)
	R2(y)	
	W2(z)	
	W2(y)	

Step (a) : We will find conflicting operations. Two operations are called as conflicting operations if all the following conditions hold true for them -

i) Both the **operations belong to different transactions**.

ii) Both the operations are on **same data item**.

iii) **At least one** of the two operations is a **write operation**



Step (b) : Now we will draw precedence graph as follows -

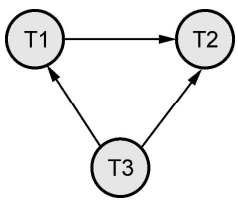


Fig. 5.5.6 Precedence graph

As there is **no cycle** in the precedence graph, the given sequence is **conflict serializable**. Hence we can convert this non serial schedule to serial schedule. For that purpose we will follow these steps to find the serializable order.

Step (c) : A **serializability order** of the transactions can be obtained by finding a linear order consistent with the partial order of the precedence graph. This process is called **topological sorting**.

Step (d) : Find the vertex which has no incoming edge which is T3. If we delete T3, then T1 is the edge that has no incoming edge. Finally find the vertex having no outgoing edge which is T2. Hence the order will be T3- T1-T2

Step 2 : We will represent the schedule S2 as follows -

T1	T2	T3
R1(x)		
	R2(z)	
		R3(x)
R1(z)		
	R2(y)	
		R3(y)
W1(x)		
	W2(z)	
		W3(y)
	W2(y)	

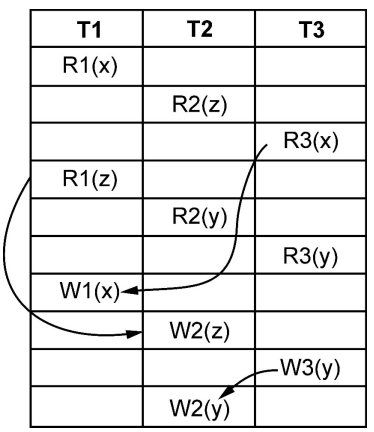
We will find **conflicting operations**. Two operations are called as conflicting operations if all the following conditions hold true for them -

i) Both the **operations belong to different transactions**.

ii) Both the operations are on **same data item**.

iii) **At least one** of the two operations is a **write operation**

The conflicting entries are as follows -



Step (b) : Now we will draw precedence graph as follows -

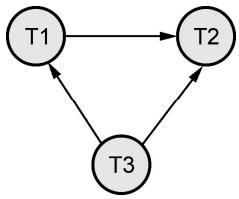


Fig. 5.5.7 Precedence Graph

As there is **no cycle** in the precedence graph, the given sequence is **conflict serializable**. Hence we can convert this non serial schedule to serial schedule. For that purpose we will follow these steps to find the serializable order.

Step (c) : A serializability order of the transactions can be obtained by finding a linear order consistent with the partial order of the precedence graph. This process is called topological sorting.

Step (d) : Find the vertex which has no incoming edge which is T3. Finally find the vertex having no outgoing edge which is T2. So in between them is T1. Hence the order will be T3- T1-T2

Example 5.5.5 *Consider the transaction, transaction and transaction are any hypothetical transactions working on data item Q. Schedule explaining the execution of and are given below. Decide whether following schedule is conflict serializable or not ? Justify your answer.*

T3	T4	T6
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

SPPU : Dec 17, End Sem, Marks 9

Solution : We will find the conflicting entries as follows -

T3	T4	T6
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

The precedence graph is as follows –

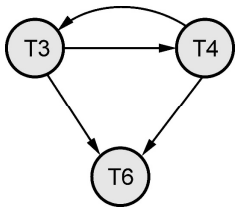


Fig. 5.5.8 Precedence Graph

As there exists **cycle**, the given schedule is **not conflict serializable**.

Example 5.5.6 *Explain the concept of conflict serializability. Decide whether following schedule is conflict serializable or not. Justify your answer.*

T1	T2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

SPPU : May 18, End Sem, Marks 9

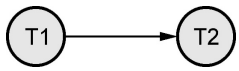
Solution :

Step 1 : We will read from top to bottom, and build a precedence graph for conflicting entries.

The conflicting entries are as follows –

T1	T2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Step 2 : Now we will build precedence graph as follows –



Step 3 : There is **no cycle** in the precedence graph. That means this schedule is conflict serializable. Hence we can convert this non serial schedule to serial schedule. For that purpose we will follow the following steps to find the serializable order.

1) Find the vertex which has no incoming edge which is T1.
2) Then find the vertex having no outgoing edge which is T2. In between them there is no other transaction.
3) Hence the order will be T1-T2

5.5.2 View Serializability

- If a given schedule is found to be view equivalent to some serial schedule, then it is called as a **view serializable schedule**.
- **View Equivalent Schedule :** Consider two schedules S1 and S2 consisting of transactions T1 and T2 respectively, then schedules S1 and S2 are said to be view equivalent schedule if it satisfies following three conditions :
  - If transaction T1 reads a data item A **from** the database initially in schedule S2, then in schedule S2 also, T1 must perform the initial read of the data item X from the database. This is same for all the data items. In other words - the initial reads must be same for all data items.
  - If data item A has been updated at last by transaction Ti in schedule S1, then in schedule S2 also, the data item A must be updated at last by transaction Ti.
  - If transaction Ti reads a data item that has been updated by the transaction Tj in schedule S1, then in schedule S2 also, transaction Ti must read the same data item that has been updated by transaction Tj. In other words the Write-Read sequence must be same.
  - **Difference between conflict serializability and view serializability**

Conflict serializability	View serializability
Every conflict serializable is view serializable.	Every view serializable schedule is not necessarily conflict serializable.
It is easy to test conflict serializability.	It is complex to test view serializability.

Steps to check whether the given schedule is view serializable or not

Step 1 : If the schedule is conflict serializable then it is surely view serializable because conflict serializability is a restricted form of view serializability.

Step 2 : If it is not conflict serializable schedule then check whether there exist any blind write operation. The blind write operation is a write operation without reading a value. If there does not exist any blind write then that means the given schedule is not view serializable. In other words if a blind write exists then that means schedule may or may not be view conflict.

Step 3 : Find the view equivalence schedule

Example 5.5.7 *Consider the following schedules for checking if these are view serializable or not.*

T_1	T_2	T_3
		W(C)
	R(A)	
	W(B)	R(B)
R(C)		
		W(B)
W(B)		

Solution :

i) The initial read operation is performed by T_2 on data item A or by T_1 on data item C. Hence we will begin with T_2 or T_1. We will choose T_2 at the beginning.

ii) The final write is performed by T_1 on the same data item B. Hence T_1 will be at the last position.

iii) The data item C is written by T_3 and then it is read by T_1. Hence T_3 should appear before T_1. Thus we get the order of schedule of view serializability as $T_2 - T_1 - T_3$

Example 5.5.8 *Consider the following schedules for checking if these are view serializable or not.*

```
T1 : read(A)
     read(B)
     if A=0 then B:=B+1;
     write(B)
T2 : read(B);
     read(A);
     if B=0 then A:=A+1;
     write(A)
```

Let consistency requirement be A=0 V B=0 with A=B=0 the initial values.

1) *Show that every serial execution involving these two transactions preserves the consistency of the Database ?*
2) *Show a concurrent execution of T1 and T2 that produces a non serializable schedule ?*
3) *Is there a concurrent execution of T1 and T2 that produces a serializable schedule ?*

Solution : 1) There are two possible executions : T_1 -> T_2 or T_2->T_1

Consider case T_1->T_2 then

A	B
0	0
0	1
0	1

$A \vee B = A$ OR $B = F \vee T = T$. This means consistency is met.

Consider case T_2 -> T_1 then

A	**B**
0	0
1	0
1	0

$A \vee B = A$ OR $B = F \vee T = T$. This means consistency is met.

2) The concurrent execution means interleaving of transactions T_1 and T_2. It can be

T_1	T_2
R(A)	
	R(B)
	R(A)
R(B) If A=0 then B=B+1	If B=0 then A=A+1 W(A)
W(B)	

This is a non-serializable schedule.

3) There is no concurrent execution resulting in a serializable schedule.

Example 5.5.9 *Consider the following schedules. The actions are listed in the order they are scheduled, and prefixed with the transaction name.*

S1 : T1 : R(X), T2 : R(X), T1 : W(Y), T2 : W(Y) T1 : R(Y), T2 : R(Y)

S2 : T3 : W(X), T1 : R(X), T1 : W(Y), T2 : R(Z), T2 : W(Z) T3 : R(Z)

For each of the schedules, answer the following questions :

i) What is the precedence graph for the schedule ?

ii) Is the schedule conflict-serializable ? If so, what are all the conflict equivalent serial schedules ?

iii) Is the schedule view-serializable ? If so, what are all the view equivalent serial schedules ?

Solution :

i) We will find **conflicting operations**. Two operations are called as conflicting operations if all the following conditions hold true for them -

- Both the **operations belong to different transactions**.
- Both the operations are on **same data item**.

  At least one of the two operations is a write operation

  For S1 : From above given example in the top to bottom scanning we find the conflict as
  - T_1 : W(Y), T_2 : W(Y) and
  - T_2 : W(Y), T_1 : R(Y)

Hence we will build the precedence graph. Draw the edge between conflicting transactions. For example in above given scenario, the conflict occurs while moving from T_1 : W(Y) to T_2 : W(Y). Hence edge must be from T_1 to T_2. Similarly for second conflict, there will be the edge from T_2 to T_1.

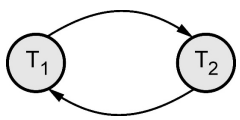


Fig. 5.5.9 Precedence graph for S1

For S_2 : The conflicts are

- T_3 : W(X), T_1 : R(X)
- T_2 : W(Z) T_3 : R(Z)

Hence the precedence graph is as follows -

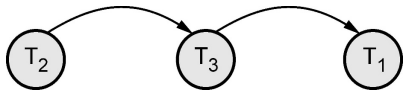


Fig. 5.5.10 Precedence graph for S_2

ii)

- S_1 is **not conflict-serializable** since the dependency **graph has a cycle**.
- S_2 is conflict-serializable as the dependency graph is acylic. The order T_2-T_3-T_1 is the only equivalent **serial order**.

iii)

- S_1 is not view serializable.
- S_2 is trivially view-serializable as it is conflict serializable. The only serial order allowed is T_2-T_3-T_1.

Example 5.5.10 *Check whether following schedule is view serializable or not. Justify your answer. (Note : T1 and T2 are transactions). Also explain the concept of view equivalent schedules and conflict equivalent schedule considering the example schedule given below*

T1	T2
read (A) A: = A - 50	
write (A)	
	read (A) temp: = A*0.1 A: = A - temp
	write (A)
read (B) B: = B + 50	
write (B)	
	read (B) B: = B + temp
	write (B)

SPPU : Dec 18,19, End Sem, Marks 9

Solution :

Step 1 : We will first find if the given schedule is conflict serializable or not. For that purpose, we will find the conflicting operations. These are as shown below -

T1	T2
read (A) A : = A – 50	
write (A)	
	read (A) temp : = A*0.1 A : = A – temp
	write (A)
read (B) B : = B + 50	
write (B)	
	read (B) B : = B + temp
	write (B)

The precedence graph is as follows –

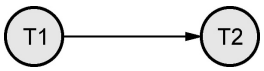


Fig. 5.5.11 Precedence graph

As there exists no cycle, the schedule is conflict serializable. The possible serializability order can be T1-T2

Now we check it for view serializability. As we get the serializability order as T1 – T2, we will find the view equivalence with the given schedule as serializable schedule.

Let S be the given schedule as given in the problem statement. Let the serializable schedule is S′={T1,T2}. These two schedules are represented as follows

T1	T2
read (A) A: = A - 50	
write (A)	
	read (A) temp: = A*0.1 A: = A - temp
	write (A)
read (B) B: = B + 50	
write (B)	
	read (B) B: = B + temp
	write (B)

Schedule S

T1	T2
read (A) A: = A - 50	
write (A)	
read (B) B: = B + 50	
write (B)	
	read (A) temp: = A*0.1 A: = A - temp
	write (A)
	read (B) B: = B + temp
	write (B)

Schedule S′

Now we will check the equivalence between them using following conditions –

1. **Initial Read**

   In **schedule S** initial read on A is in transaction T1. Similarly initial read on B is in transaction T1.

   Similarly in **schedule S′**, initial read on A is in transaction T1. Similarly initial read on B is in transaction T1.

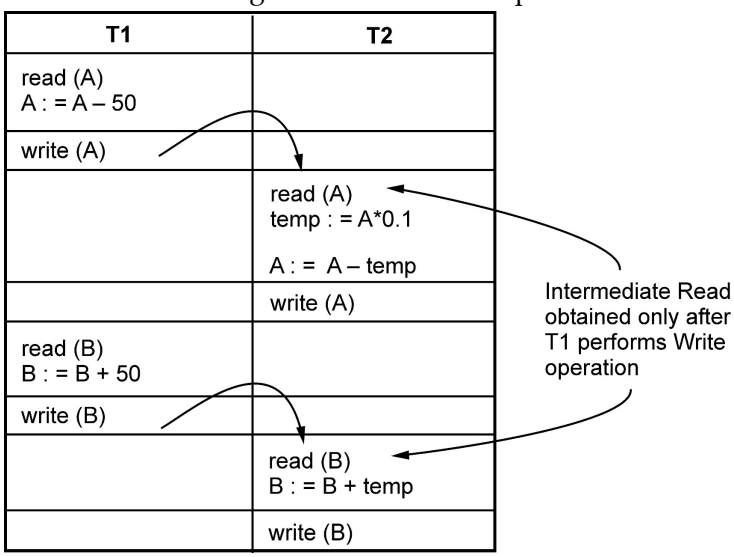
2. **Final Write**

   In **schedule S** final write on A is in transaction T2. Similarly final write on B is in transaction T2.

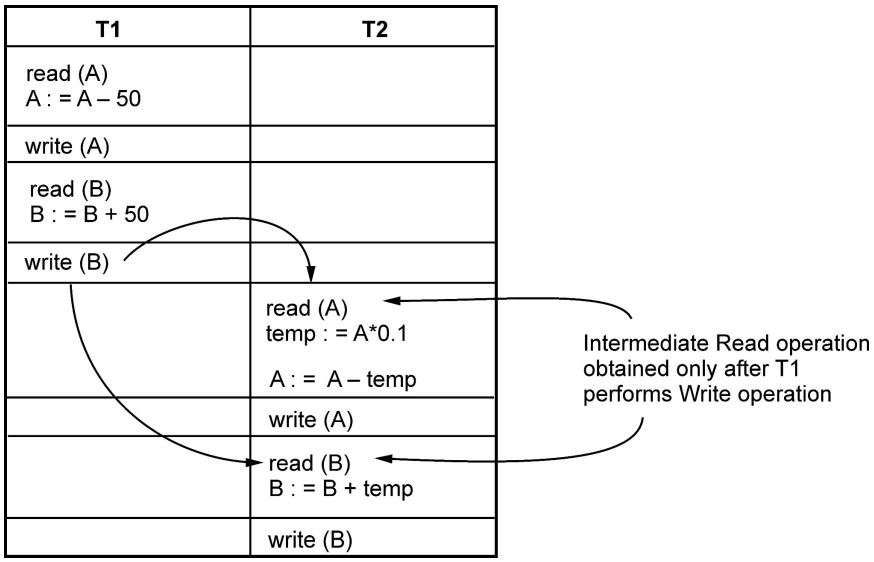
   In **schedule S′** final write on A is in transaction T2. Similarly final write on B is in transaction T2

3. **Intermediate Read**

   Consider **schedule S** for finding intermediate read operation.



Similarly consider **schedule S′** for finding intermediate read operation.



In both the **schedules S** and **S′**, the intermediate read operation is performed by T2 only after T1 performs write operation.

Thus all the above three conditions get satisfied. Hence given schedule is **view serializable**.

Review Question

1. *Explain the concept of conflict serializability with example. Since every conflict-serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability ?* **SPPU : Dec 18,19, End Sem, Marks 8**

5.6 Recoverable and Non-recoverable Schedules

- The serializable schedule can be made consistent by applying conflict serializability or view serializability.
- The serializable order makes a transaction isolation. But during the execution of concurrent transactions in a given schedule, some of the transaction may get failed(may be due to hardware or software failure)
- If the transaction gets failed we need to undo the effect of this transaction to ensure the atomicity property of transaction. This makes the schedule acceptable even after the failure.
- The schedule can be made acceptable by two techniques namely - Recoverable Schedule and Cascadeless schedule.

5.6.1 Recoverable Schedule

Definition : A recoverable schedule is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the commit operation of T_j.

For example : Consider following schedule, consider A = 100

T_1	T_2
R(A)	
A=A+50	
W(A)	
	R(A)
	A=A–20
	W(A)
	Commit
Some transaction…	
Commit	

Failure

- The above schedule is inconsistent if failure occurs after the commit of T_2.
- It is because T_2 is **dependable transaction** on T_1. A transaction is said to be dependable if it contains a **dirty read.**
- The dirty read is a situation in which one transaction reads the data immediately after the write operation of previous transaction.

T_1	T_2
R(A)	
A=A+50	
W(A)	
	R(A)
	A=A–20
	W(A)
	Commit
Commit	

Dirty read

- Now if the dependable transaction i.e. T_2 is committed first and then failure occurs then if the transaction T_1 makes any changes then those changes will not be known to the T_2. This leads to non recoverable state of the schedule.
- To make the schedule recoverable we will apply the rule that - commit the independent transaction before any dependable transaction.
- In above example independent transaction is T_1, hence we must commit it before the dependable transaction i.e. T_2.
- The recoverable schedule will then be -

T_1	T_2
R(A)	
A=A+50	
W(A)	
	R(A)
	A=A–20
	W(A)
Commit	
	Commit

5.6.2 Cascadeless Schedule

Definition : If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written that data item is committed or aborted, then such a schedule is known as a cascadeless schedule.

The cascadeless schedule allows only **committed Read** operation. For example :

T_1	T_2	T_3
R(A)		
A=A+50		
W(A)		
Commit		
	R(A)	
	A=A–20	
	W(A)	
	Commit	
		R(A)
		W(A)

In above schedule at any point if the failure occurs due to commit operation before every Read operation of each transaction, the schedule becomes recoverable and atomicity can be maintained.

Part II : Concurrency Control

5.7 Concurrency Control

- One of the fundamental properties of a transaction is **isolation.**
- When several transactions execute concurrently in the database, however, the isolation property may no longer be preserved.
- A database can have multiple transactions running at the same time. This is called **concurrency.**
- To preserve the isolation property, the system must control the interaction among the concurrent transactions; this control is achieved through one of a variety of mechanisms called **concurrency control schemes.**
- **Definition of concurrency control :** A mechanism which ensures that simultaneous execution of more than one transactions does not lead to any database inconsistencies is called **concurrency control mechanism.**
- The concurrency control can be achieved with the help of various protocols such as - lock based protocol, deadlock handling, multiple granularity, timestamp based protocol, and validation based protocols.

5.8 Need for Concurrency

- Following are the purposes of concurrency control –
  - **To ensure isolation**
  - **To resolve read-write or write-write conflicts**
  - **To preserve consistency of database**
- Concurrent execution of transactions over shared database creates several data integrity and consistency problems - these are

1) Lost Update Problem : This problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.

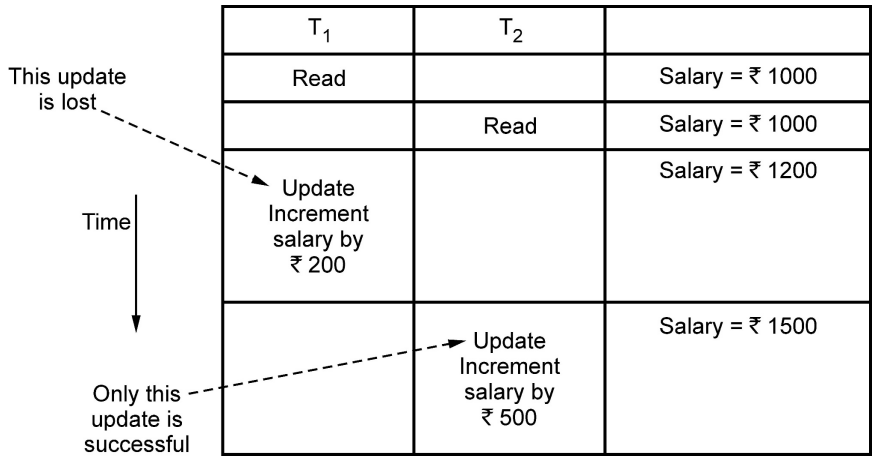
For example - Consider following transactions

1) Salary of employee is read during transaction T1.

2) Salary of employee is read by another transaction T_2.

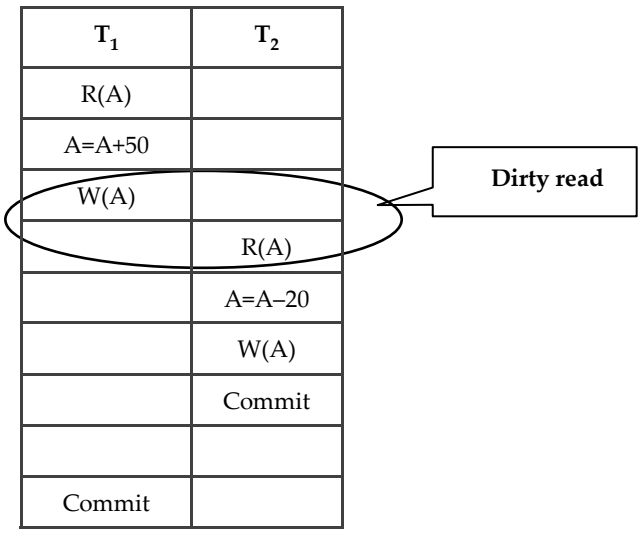
3) During transaction T_1, the salary is incremented by ₹ 200

4) During transaction T_2, the salary is incremented by ₹ 500



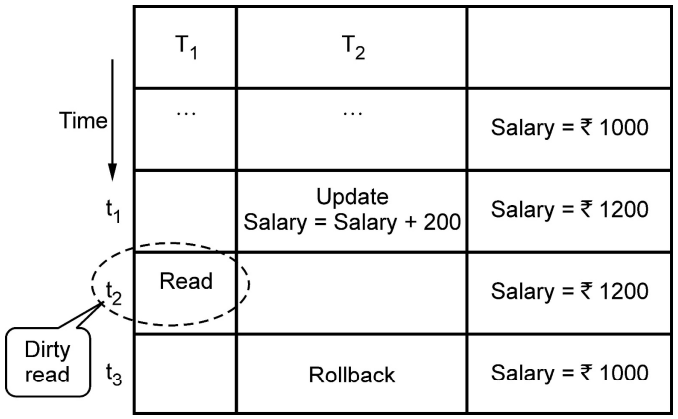
The result of the above sequence is that the update made by transaction T1 is completely lost. Therefor this problem is called as lost update problem.

2) Dirty Read or Uncommited Read Problem : The dirty read is a situation in which one transaction reads the data immediately after the write operation of previous transaction



For example - Consider following transactions -

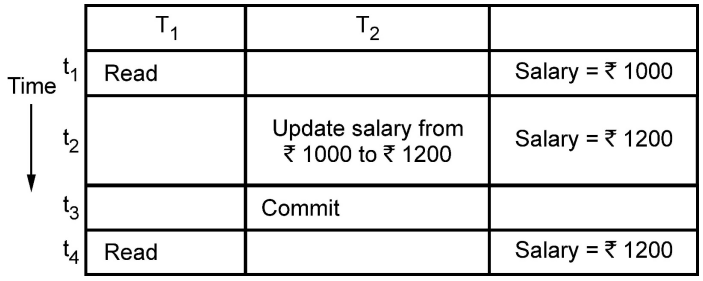
Assume initially salary is = ₹ 1000



1) At the time t1, the transaction T_2 updates the salary to ₹ 1200
2) This salary is read at time t_2 by transaction T_1. Obviously it is ₹ 1200
3) But at the time t_3, the transaction T_2 performs Rollback by undoing the changes made by T_1 and T_2 at time t1 and t_2.
4) Thus the salary again becomes = ₹ 1000. This situation leads to **Dirty Read or Uncommited Read** because here the read made at time t_2 **(immediately after update of another transaction)** becomes a dirty read.

3) Non-repeatable read problem

This problem is also known as **inconsistent analysis problem.** This problem occurs when a particular transaction sees two different values for the same row within its lifetime. For example –



1) At time t_1, the transaction T_1 reads the salary as ₹ 1000
2) At time t_2 the transaction T_2 reads the same salary as ₹ 1000 and updates it to ₹1200
3) Then at time t_3, the transaction T_2 gets committed.
4) Now when the transaction T_1 reads the same salary at time t_4, it gets different value than what it had read at time t_1. Now, transaction T_1 cannot repeat its reading operation. Thus inconsistent values are obtained.

Hence the name of this problem is non-repeatable read or inconsistent analysis problem.

4) Phantom read problem

The phantom read problem is a special case of non repeatable read problem.

This is a problem in which one of the transaction makes the changes in the database system and due to these changes another transaction can not read the data item which it has read just recently. For example –

Time	T_1	T_2	
t_1	Read		Salary = ₹ 1000
t_2		Read	Salary = ₹ 1000
t_3	Delete salary		No salary
t_4		Read	"Can not find salary"

1) At time t_1, the transaction T_1 reads the value of salary as ₹ 1000
2) At time t_2, the transaction T_2 reads the value of the same salary as ₹ 1000
3) At time t_3, the transaction T_1 deletes the variable salary.
4) Now at time t_4, when T_2 again reads the salary it gets error. Now transaction T_2 can not identify the reason why it is not getting the salary value which is read just few time back.

This problem occurs due to changes in the database and is called phantom read problem.

5.9 Lock based Protocol

SPPU : Dec.-17,18,19, May-18,19, Marks 9

5.9.1 Why Do We Need Lock ?

- One of the method to ensure the **isolation** property in transactions is to require that data items be accessed in a mutually exclusive manner. That means, while one transaction is accessing a data item, no other transaction can modify that data item.
- The most common method used to implement this requirement is to allow a transaction to access a data item only if it is currently holding a **lock on that item.**
- Thus the lock on the operation is required to ensure the isolation of transaction.

5.9.2 Working of Lock

- **Concept of protocol :** The lock based protocol is a mechanism in which there is exclusive use of **locks** on the data item for current transaction.

- **Types of locks :** There are two types of locks used –

Fig. 5.9.1 Types of locks

i) **Shared lock :** The shared lock is used for reading data items only. It is denoted by Lock-S. This is also called as **read lock**.

ii) **Exclusive lock :** The exclusive lock is used for both read and write operations. It is denoted as Lock-X. This is also called as **write lock**.

- The **compatibility matrix** is used while working on set of locks. The **concurrency control manager** checks the compatibility matrix before granting the lock. If the two modes of transactions are compatible to each other then only the lock will be granted.
- In a set of locks may consists of shared or exclusive locks. Following matrix represents the compatibility between modes of locks.

	S	X
S	T	F
X	F	F

Fig. 5.9.2 Compatibility matrix for locks

Here T stands for True and F stands for False. If the control manager get the compatibility mode as True then it grant the lock otherwise the lock will be denied.

- **For example :** If the transaction T_1 is holding a shared lock in data item A, then the control manager can grant the shared lock to transaction T_2 as compatibility is True.

  But it cannot grant the exclusive lock as the compatibility is false. In simple words if transaction T_1 is reading a data item A then same data item A can be read by another transaction T_2 but cannot be written by another transaction.

- Similarly if an exclusive lock (i.e. lock for read and write operations) is hold on the data item in some transaction then no other transaction can acquire shared or exclusive lock as the compatibility function denotes F. That means of some transaction is writing a data item A then another transaction can not read or write that data item A.

- Hence the **rule of thumb is**

i) **Any number** of transactions can hold **shared lock** on an item.

ii) But **exclusive lock can be hold by only one transaction**.

- **Example of a schedule denoting shared and exclusive locks :** Consider following schedule in which initially A=100. We deduct 50 from A in T1 transaction and Read the data item A in transaction T2. The scenario can be represented with the help of locks and concurrency control manager as follows :

Exclusive lock

Shared lock

T_1	T_2	Concurrency control manager
Lock-X(A)		
		Grant X(A,T_1) because in T_1 there is write operation.
R(A)		
A=A – 50		
W(A)		
Unlock(A)		
	Lock-S(A)	
		Grant S(A,T_2) because in T_2 there is read operation
	R(A)	
	Unlock(A)	

5.9.3 Two Phase Locking Protocol

- The two phase locking is a protocol in which there are two phases :

  i) Growing phase (Locking phase) : It is a phase in which the transaction may obtain locks but does not release any lock.

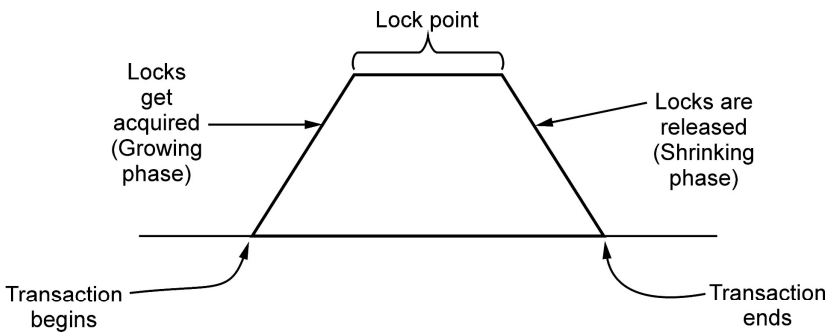
  ii) Shrinking phase (Unlocking phase) : It is a phase in which the transaction may release the locks but does not obtain any new lock.

- **Lock Point :** The **last lock position** or **first unlock position** is called lock point.

- For example -

Lock(A)
Lock(B)
Lock(C)
….
…
…
Unlock(A)
Unlock(B)
Unlock(C)

Lock Point



Consider following transactions

T_1	T_2
Lock-X(A)	Lock-S(B)
Read(A)	Read(B)
A=A–50	Unlock-S(B)
Write(A)	
Lock-X(B)	
Unlock-X(A)	
B=B+100	Lock-S(A)
Write(B)	Read(A)
Unlock-X(B)	Unlock-S(A)

The important rule for being a two phase locking is - **All lock operations precede all the unlock operations**.

In above transactions T_1 is in two phase locking mode but transaction T_2 is not in two phase locking. Because in T_2, the shared lock is acquired by data item B, then data item B is read and then the lock is released. Again the lock is acquired by data item A , then the data item A is read and the lock is then released. Thus we get lock-unlock-lock-unlock sequence. Clearly this is not possible in two phase locking.

Example 5.9.1 *Prove that two phase locking guarantees serializability.*

Solution :

- Serializability is mainly an issue of handling write operation. Because any inconsistency may only be created by **write** operation.
- Multiple reads on a database item can happen parallely.
- 2-Phase locking protocol restricts this unwanted read/write by applying **exclusive lock**.
- Moreover, when there is an **exclusive lock** on an item it will **only be released in shrinking phase**. Due to this restriction there is no chance of getting any inconsistent state.

The serializability using two phase locking can be understood with the help of following example :

Consider two transactions

T_1	T_2
R(A)	
	R(A)
R(B)	
W(B)	

Step 1 : Now we will **apply two phase locking**. That means we will **apply locks in growing** and shrinking phase

T_1	T_2
Lock-S(A)	
R(A)	
	Lock-S(A)
	R(A)
Lock-X(B)	
R(B)	
W(B)	
Unlock-X(B)	
	Unlock-S(A)

Note that above schedule is serializable as it prevents interference between two transactions.

The serializability order can be obtained based on the **lock point**. The lock point is either last lock operation position or first unlock position in the transaction.

The last lock position is in T_1, then it is in T_2. Hence the serializability will be T_1->T_2 based on lock points. Hence The **serializability sequence** can be $\mathbf{R_1(A);R_2(A);R_1(B);W_1(B)}$

Advantages of two phase locking

(1) It ensures serializability.

Disadvantages of two phase locking protocol

(1) It leads to dealocks.

(2) It leads to cascading rollback.

Problems in two phase locking

The two phase locking protocol leads to two problems - Deadlock and cascading roll back.

1) **Deadlock :** The deadlock problem can not be solved by two phase locking. Deadlock is a situation in which when two or more transactions have got a lock and waiting for another locks currently held by one of the other transactions.

For example

T_1	T_2
Lock-X(A)	Lock-X(B)
Read(A)	Read(B)
A=A–50	B=B+100
Write(A)	Write(B)
Delayed, wait for T_2 to release Lock on B	Delayed, wait for T_1 to release Lock on A

2) **Cascading Rollback :** Cascading rollback is a situation in which a single transaction failure leads to a series of transaction rollback. For example -

T_1	T_2	T_3
Read(A)		
Read(B)		
C=A+B		
Write(C)		
	Read(C)	
	Write(C)	
		Read(C)

When T_1 writes value of C then only T_2 can read it. And when T_2 writes the value of C then only transaction T_3 can read it. But if the transaction T_1 gets failed then automatically transactions T_2 and T_3 gets failed.

The simple two phase locking does not solve the cascading rollback problem. To solve the problem of cascading Rollback two types of two phase locking mechanisms can be used.

5.9.3.1 Types of Two Phase Locking

1) **Strict two phase locking :** The strict 2PL protocol is a basic two phase protocol **but all the exclusive mode locks** be **held until the transaction commits**. That means in other words all the **exclusive locks** are **unlocked** only after the **transaction is committed**. That also means that if T_1 has exclusive lock, then T_1 will release the exclusive lock only after commit operation, then only other transaction is allowed to read or write. For example - Consider two transactions

T_1	T_2
W(A)	
	R(A)

If we apply the locks then

T_1	T_2
Lock-X(A)	
W(A)	
Commit	
Unlock(A)	
	Lock-S(A)
	R(A)
	Unlock-S(A)

Thus only after commit operation in T_1, we can unlock the exclusive lock. This ensures the strict serializability.

Thus compared to basic two phase locking protocol, the advantage of strict 2PL protocol is it ensures strict serializability.

2) **Rigorous two phase locking :** This is stricter two phase locking protocol. Here all locks are to be held until the transaction commits. The transactions can be seriealized in the order in which they commit.

Example - Consider transactions

T_1
R(A)
R(B)
W(B)

If we apply the locks then

T_1
Lock-S(A)
R(A)
Lock-X(B)
R(B)
W(B)
Commit
Unlock(A)
Unlock(B)

Thus the above transaction uses rigorous two phase locking mechanism.

Example 5.9.2 *Consider the following two transactions :*
T_1 : read(A)Read(B);
If A=0 then B=B+1;
Write(B)
T_2 : read(B); read(A)
If B=0 then A=A+1
Write(A)
Add lock and unlock instructions to transactions T_1 and T_2, so that they observe two phase locking protocol. Can the execution of these transactions result in deadlock ?

Solution :

T_1	T_2
Lock-S(A)	Lock-S(B)
Read(A)	Read(B)
Lock-X(B)	Lock-X(A)
Read(B)	Read(A)
if A=0 then B=B+1	if B=0 then A=A+1
Write(B)	Write(A)
Unlock(A)	Unlock(B)
Commit	Commit
Unlock(B)	Unlock(A)

This is lock-unlock instruction sequence help to satisfy the requirements for strict two phase locking for the given transactions.

The execution of these transactions result in deadlock. Consider following partial execution scenario which leads to deadlock.

T_1	T_2
Lock-S(A)	Lock-S(B)
Read(A)	Read(B)
Lock-X(B)	Lock-X(A)
Now it will wait for T_2 to release exclusive lock on A	Now it will wait for T_1 to release exclusive lock on B

Review Questions

1. *A transaction may be waiting for more time for an Exclusive (X) lock on an item, while a sequence of other transactions request and are granted as Shared (S) lock on the same item. What is this problem ? How it is solved by two phase lock protocol ?*
   SPPU : Dec.-17, End Sem,Marks 8
2. *Explain the two phase lock protocol for concurrency control. Also explain its two versions : strict two phase lock protocol and rigorous two phase lock protocol.*
   SPPU : May-18, End Sem, Marks 8
3. *Explain the Two Phase lock Protocol and show how it ensures conflict serializability. Two Phase lock protocol does not ensure freedom from deadlock explain with necessary example. Also explain its two versions : strict two phase lock protocol and rigorous two phase lock protocol.* **SPPU : Dec.-18, 19, End Sem, Marks 9**
4. *What benefit does rigorous two-phase locking provide ? How does it compare with other forms of two - phase locking ?* **SPPU : May-19, End Sem, Marks 9**

5.10 Time-stamp based Protocol

SPPU : May-19, Dec.-17, Marks 9

- The time stamp ordering protocol is a scheme in which the order of transaction is decided in advance based on their timestamps. Thus the schedules are serialized according to their timestamps.
- The timestamp-ordering protocol ensures that any conflicting read and write operations are executed in timestamp order.
- A larger timestamp indicates a more recent transaction or it is also called as **younger transaction** while lesser timestamp indicates **older transaction.**
- Assume a collection of data items that are accessed, with read and write operations, by transactions.
- For each data item X the DBMS maintains the following values : –
  - **RTS(X) :** The timestamp on which object X was last read (by some transaction T_i , i.e., RTS(X):=TS(T_i)) [Note that : RTS stands for Read Time Stamp]
  - **WTS(X) :** The timestamp on which object X was last written (by some transaction T_j , i.e., WTS(X):=TS(T_j)) [**Note that** : WTS stands for Write Time Stamp]
- For the following algorithms we use the following assumptions : A data item X in the database has a **RTS(X)** and **WTS(X)**. These are actually the timestamps of read and write operations performed on data item X at latest time.
- A transaction T attempts to perform some action (read or write) on data item X on some timestamp and we call that timestamp as **TS(T)**.
- By timestamp ordering algorithm we need to decide whether transaction T has to be aborted or T can continue execution.

Basic Timestamp Ordering Algorithm

Case 1 (Read) : Transaction T issues **a read(X) operation**

i) If TS(T) < WTS(X), then read(X) is rejected. T has to abort and be rejected.

ii) If WTS(X) ≤ TS(T), then execute read(X) of T and update RTS(X).

Case 2 (Write) : Transaction T issues a **write(X) operation**

i) If TS(T) < RTS(X) or if TS(T) < WTS(X), then write is rejected

ii) If RTS(X) ≤ TS(T) or WTS(X) ≤ TS(T), then execute write(X) of T and update WTS(X).

Example for Case 1 (Read operation)

(i) Suppose we have two transactions T_1 and T_2 with timestamps 10 sec. and 20 sec. respectively.

10 Sec T_1	20 Sec T_2
R(X)	
	W(X)
R(X)	

RTS(X) and WTS(X) is initially = 0

Then RTS(X) = 10, when transaction T_1 executes

After that WTS(X) = 20 when transaction T_2 executes

Now if read operation R(X) occurs on transaction T_1 at $TS(T_1)$ = 10 then

$TS(T_1)$ i.e. 10 <WTS(X) i.e. 20, hence we have to reject second read operation on T_1 i.e.

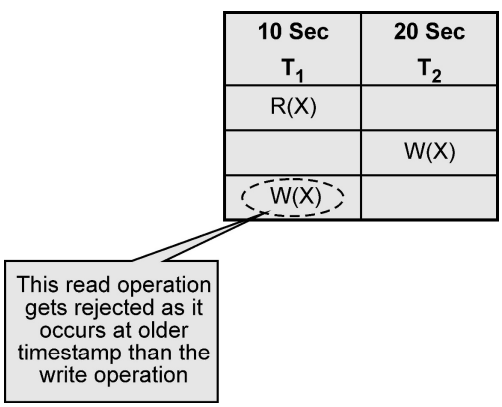


Fig. 5.10.1

ii) Suppose we have two transactions T_1 and T_2 with timestamps 10 sec. and 20 sec. respectively.

10 Sec T_1	20 Sec T_2
W(X)	
	R(X)

RTS(X) and WTS(X) is initially = 0

Then WTS(X) =10 as transaction T_1 executes.

Now if Read operation R(X) occurs on transaction T_2 at $TS(T_2)$ = 20 then

$TS(T_2)$ i.e. 20 >WTS(X) which is 10, hence we accept read operation on T_2. The transaction T_2 will perform read operation and now RTS will be updated as,

RTS(X) = 20

Example for Case 2 (Write Operation)

i) Suppose we have two transactions T_1 and T_2 with timestamps 10 sec. and 20 sec. respectively.

10 Sec. T_1	20 Sec. T_2
R(X)	
	W(X)
W(X)	

RTS(X) and WTS(X) is initially = 0

Then RTS(X) = 10, when transaction T_1 executes

After that WTS(X) = 20 when transaction T_2 executes

Now if write operation W(X) occurs on transaction T_1 at $TS(T_1)$ = 10 then

$TS(T_1)$ i.e. 10 <WTS(X), hence we have to reject second write operation on T_1 i.e.

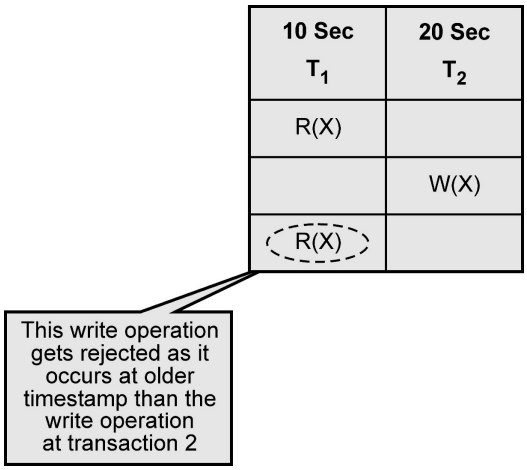


Fig. 5.10.2

ii) Suppose we have two transactions T_1 and T_2 with timestamps 10 sec. and 20 sec. respectively.

10 Sec T_1	20 Sec T_2
W(X)	
	W(X)

RTS(X) and WTS(X) is initially = 0

Then WTS(X) = 10 as transaction T_1 executes.

Now if write operation W(X) occurs on transaction T_2 at $TS(T_2) = 20$ then

$TS(T_2)$ i.e. 20 >WTS(X) which is 10, hence we accept write operation on T_2. The transaction T_2 will perform write operation and now WTS will be updated as,

WTS(X) = 20

Advantages and disadvantages of time stamp ordering

Advantages

1) Schedules are serializable
2) No waiting for transaction and hence there is no deadlock situation.

Disadvantages

1) Schedules are not recoverable once transactions occur.
2) Same transaction may be continuously aborted or restarted.

Review Question

1. *Suppose a transaction , issues a read command on data item Q. How time - stamp based protocol decides whether to allow the operation to be executed or not using time-stamp based protocol of concurrency control.* **SPPU : Dec.-17, May-19, End Sem, Marks 9**

5.11 Deadlocks Handling

Deadlock is a specific concurrency problem in which two transactions depend on each other for something.

For example - Consider that transaction T_1 holds a lock on some rows of table **A** and needs to update some rows in the **B** table. Simultaneously, transaction T_2 holds locks on some rows in the **B** table and needs to update the rows in the **A** table held by transaction T_1.

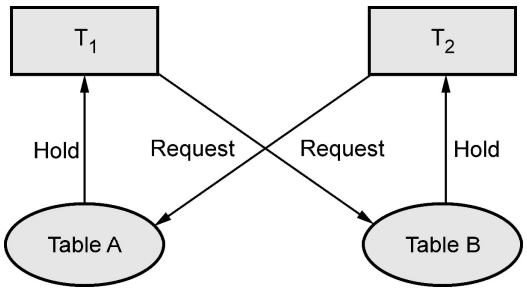


Fig. 5.11.1

Now, the main problem arises. Now transaction T_1 is waiting for T_2 to release its lock and similarly, transaction T_2 is waiting for T_1 to release its lock. All activities come to a halt state and remain at a standstill. This situation is called **deadlock** in DBMS.

Definition : Deadlock can be formally defined as - " A system is in deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set. "

There are **four conditions for a deadlock to occur**

A deadlock may occur if all the following conditions holds true.

1. **Mutual exclusion condition :** There must be at least one resource that cannot be used by more than one process at a time.
2. **Hold and wait condition :** A process that is holding a resource can request for additional resources that are being held by other processes in the system.
3. **No preemption condition :** A resource cannot be forcibly taken from a process. Only the process can release a resource that is being held by it.
4. **Circular wait condition :** A condition where one process is waiting for a resource that is being held by second process and second process is waiting for third process

….so on and the last process is waiting for the first process. Thus making a circular chain of waiting.

Deadlock can be handled using two techniques -

1. Deadlock prevention 2. Deadlock detection and deadlock recovery

1. Deadlock prevention :

For large database, deadlock prevention method is suitable. A deadlock can be prevented if the resources are allocated in such a way that deadlock never occur. The DBMS analyzes the operations whether they can create deadlock situation or not, If they do, that transaction is never allowed to be executed.

There are two techniques used for deadlock prevention -

i) Wait-Die :

- In this scheme, if a transaction requests for a resource which is already held with a conflicting lock by another transaction then the DBMS simply checks the timestamp of both transactions. It **allows the older transaction to wait** until the resource is available for execution.
- Suppose there are two transactions T_i and T_j and let TS(T) is a timestamp of any transaction T. If T_2 holds a lock by some other transaction and T_1 is requesting for resources held by T_2 then the following actions are performed by DBMS :
  - Check if $TS(T_i) < TS(T_j)$ - If T_i is the older transaction and T_j has held some resource, then T_i is allowed to wait until the data-item is available for execution. That means if the older transaction is waiting for a resource which is locked by the younger transaction, then the older transaction is allowed to wait for resource until it is available.
  - Check if $TS(T_i) < TS(T_j)$ - If T_i is older transaction and has held some resource and if Tj is waiting for it, then T_j is killed and restarted later with the random delay but with the same timestamp.

Timestamp is a way of assigning priorities to each transaction when it starts. If timestamp is lower then that transaction has higher priority. **That means oldest transaction has highest priority**.

For example -

Let T_1 is a transaction which requests the data item acquired by transaction T_2. Similarly T_3 is a transaction which requests the data item acquired by transaction T_2.

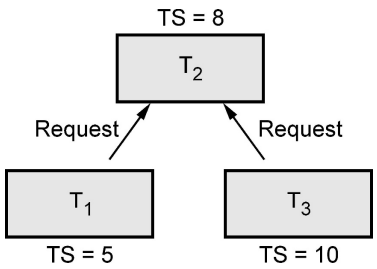


Fig. 5.11.2

Here $TS(T_1)$ i.e. Time stamp of T_1 is less than $TS(T_3)$. In other words T_1 is older than T_3 . Hence T_1 is made to **wait** while T_3 is **rolledback**.

ii) Wound- wait :

- In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After some delay, the younger transaction is restarted but with the same timestamp.
- If the older transaction has held a resource which is requested by the younger transaction, then the younger transaction is asked to wait until older releases it.

Suppose T_1 needs a resource held by T_2 and T_3 also needs the resource held by T_2, with $TS(T_1) = 5$, $TS(T_2) = 8$ and $TS(T_3) = 10$, then T_1 being older waits and T_3 being younger dies. After the some delay, the younger transaction is restarted but with the same timestamp.

This ultimately prevents a deadlock to occur.

To summarize

	Wait-Die	**Wound-wait**
Older transaction needs a data item held by younger transaction	Older transaction waits	Younger transaction dies.
Younger transaction needs a data item held by older transaction	Younger transaction dies	Younger transaction dies.

2. Deadlock detection :

- In deadlock detection mechanism, an algorithm that examines the state of the system is invoked periodically to determine whether a deadlock has occurred or not. If deadlock is occurrence is detected, then the system must try to recover from it.

- Deadlock detection is done using wait for graph method.

Wait for graph

- In this method, a graph is created based on the transaction and their lock. If the created **graph has a cycle or closed loop, then there is a deadlock**.
- The wait for the graph is maintained by the system for every transaction which is waiting for some data held by the others. The system keeps checking the graph if there is any cycle in the graph.
- This graph consists of a pair G = (V, E), where V is a set of vertices and E is a set of edges.
- The set of vertices consists of all the transactions in the system.
- When transaction T_i requests a data item currently being held by transaction T_j, then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This edge is removed only when transaction T_j is no longer holding a data item needed by transaction T_i.

For example - Consider following transactions, We will draw a wait for graph for this scenario and check for deadlock.

T_1	T_2
R(A)	
	R(A)
W(A)	
R(B)	
	W(A)
W(B)	

We will use three rules for designing the wait-for graph -

Rule 1 : If T_1 has **Read** operation and then T_2 has **Write** operation then draw an edge T_1->T_2.

Rule 2 : If T_1 has **Write** operation and then T_2 has **Read** operation then draw an edge T_1->T_2.

Rule 3 : If T_1 has **Write** operation and then T_2 has **Write** operation then draw an edge T_1->T_2.

Let us draw wait-for graph

Step 1 : Draw vertices for all the transactions

Fig. 5.11.3

Step 2 : We find the Read-Write pair from two different transactions reading from top to bottom. If such as pair is found then we will add the edges between corresponding directions. For instance -

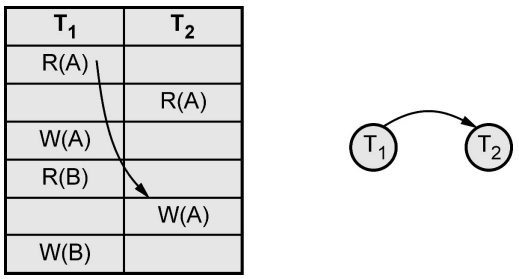


Fig. 5.11.4

Step 3 :

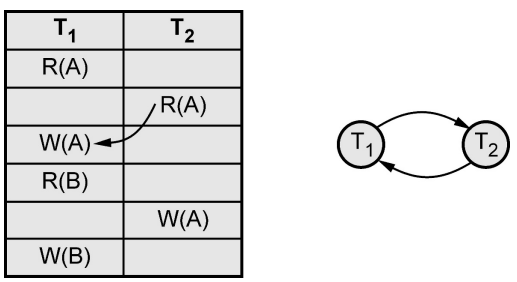


Fig. 5.11.5

As **cycle is detected** in the wait-for graph there is no need to further process. The **deadlock** is present in this transaction scenario.

Example 5.11.1 *Give an example of a scenario where two phase locking leads to deadlock.*

Solution : Following scenario of execution of transactions can result in deadlock.

T_1	T_2
Lock – S (A)	
	Lock – S (B)
	Read (B)
Read (A)	
Lock – X (B)	
	Lock – X (A)

These two instructions cause a deadlock situation.

In above scenario, transaction T_1 makes an exclusive lock on data item B and then transaction T_2 makes an exclusive lock on data item A. Here unless and until T_1 does not give up the lock (i.e. unlock) on B; T_2 cannot read / write it. Similarly unless and until T_2 does not give up the lock on A; T_1 cannot read or write on A.

This is a purely deadlock situation in two phase locking.

Example 5.11.2 *What is deadlock ? Consider following sequence of actions listed in order they are submitted to DBMS*
Sequence : S1:R1(A)W2(B);R1(B);R3(C);w2(c);W4(B);W3(A)
Draw waits-for graph in case of deadlock situation

Solution :

Step 1 : We will rewrite the schedule as,

T1	T2	T3	T4
R1(A)			
	W2(B)		
R1(B)			
		R3(C)	
	W2(C)		
			W4(B)
		W3(A)	

Step 2 :

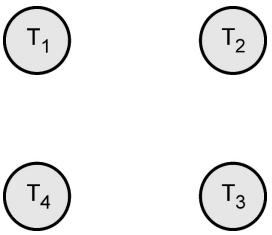
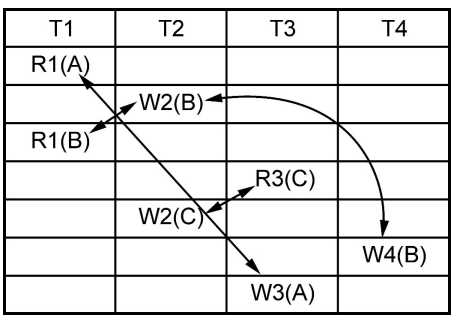


Fig. 5.11.6

As there are four transactions, there will be four nodes.

Step 3 : We find the Read-Write pair from two different transactions reading from top to bottom. If such as pair is found then we will add the edges between corresponding directions. For instance -



Hence the wait-for graph will be,

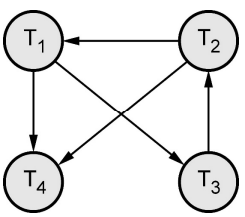


Fig. 5.11.7

The cycle is detected in wait-for graph as T1 → T3 → T2. That means deadlock is present in this transaction scenario.

5.12 Recovery Concepts

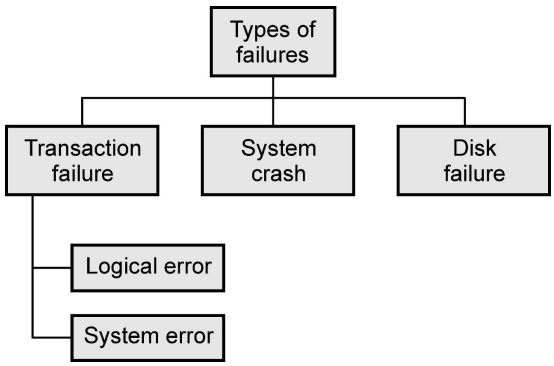
5.12.1 Purpose of Database Recovery

- The purpose of recovery is to bring the database into the last consistent stage prior to occurrence of failure.

- The recovery must preserve all the ACID properties of transaction. The ACID properties are - Atomicity, consistency, isolation and durability.
- Thus recovery ensures high availability of the database for transaction purpose.
- For example - If the system crashes before the amount transfer from one account to another then either one or both the accounts may have incorrect values. Here the database must be recovered before the modification takes place.

5.12.2 Failure Classification

Various types of failures are –



1) Transaction failure : Following are two types of errors due to which the transaction gets failed.

- **Logical error :**

i) This error is caused due to internal conditions such as bad input, data not found, overflow of resource limit and so on.

ii) Due to logical error the transaction can not be continued.

- **System error :**

i) When the system enters in an undesired state and then the transaction can not be continued then this type of error is called as system error.

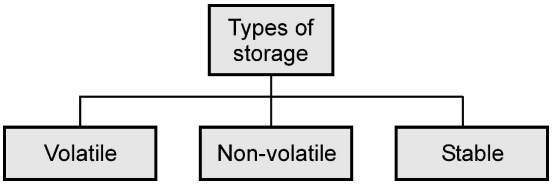
2) System crash : The situation in which there is a hardware malfunction, or a bug in the database software or the operating system, and because of which there is a loss of the content of volatile storage, and finally the transaction processing come to a halt is called system crash.

3) Disk failure : A disk block loses its content as a result of either a head crash or failure during a data-transfer operation. The backup of data is maintained on the secondary disks or DVD to recover from such failure.

5.12.3 Storage

A DBMS stores the data on external storage because the amount of data is very huge and must persist across program executions.

The storage structure is a memory structure in the system. It has following categories –



1) Volatile :

- Volatile memory is a primary memory in the system and is placed along with the CPU.
- These memories can store only small amount of data, but they are very fast. For example - main memory, cache memory.
- A volatile storage cannot survive system crashes.
- That means data in these memories will be lost on failure.

2) Non volatile :

- Non volatile memory is a secondary memory and is huge in size. For example : Hard disk, Flash memory, Magnetic tapes.
- These memories are designed to withstand system crashes.

3) Stable :

- Information residing in stable storage is never lost.
- To implement stable storage, we replicate the information in several nonvolatile storage media (usually disk) with independent failure modes.

Stable Storage Implementation

- Stable storage is a kind of storage on which the information residing on it is never lost.
- Although stable storage is theoretically impossible to obtain it can be approximately built by applying a technique in which **data loss is almost impossible.**
- That means the information is **replicated** in several nonvolatile storage media with **independent failure modes.**
- Updates must be done with care to ensure that a failure during an update to stable storage does not cause a loss of information.

5.13 Recovery Methods

Following are commonly used crash recovery methods –

1) Shadow paging
2) Log-based recovery
   a. Deferred Update
   b. Immediate Update
3) Checkpointing

5.14 Shadow-paging

- Shadow paging is a recovery scheme in which database is considered to be made up of number of fixed size disk pages.
- A directory or a page table is constructed with n number of pages where each ith page points to the i^{th} database page on the disk. (Refer Fig. 5.14.1)

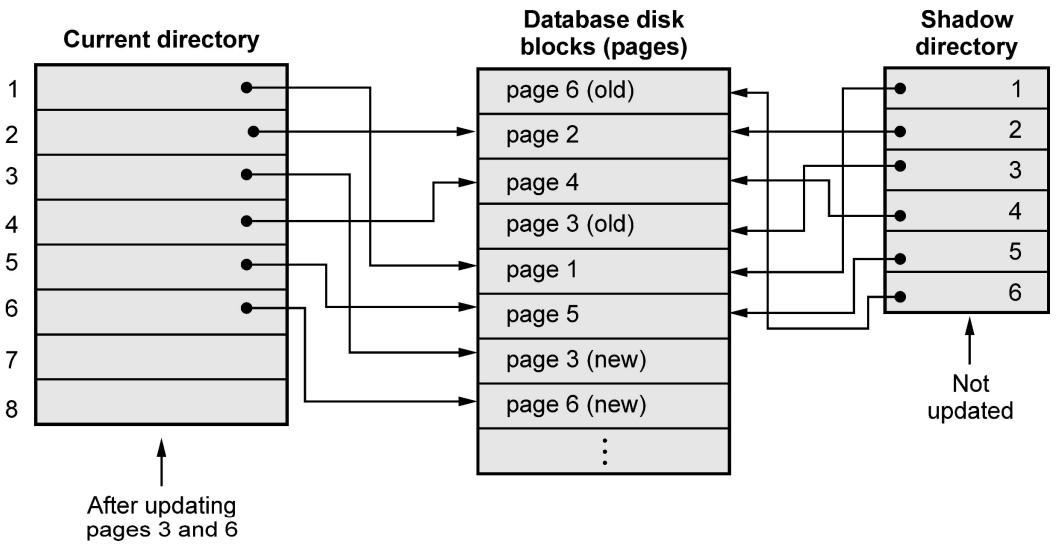


Fig. 5.14.1 Demonstration of shadow paging

- The directory can be kept in the main memory.
- When a transaction begins executing, the current directory-whose entries point to the most recent or current database pages on disk-is copied into a another directory called shadow directory.
- The shadow directory is then saved on disk while the current directory is used by the transaction.
- During the execution of transaction, the shadow directory is never modified.

- When a write operation is to be performed then the **new copy** of modified database page is created but the old copy of database page is never overwritten. This newly created database page is written somewhere else.
- The current directory will point to newly modified web page and the shadow page directory will point to the old web page entries of database disk.
- When the failure occurs then the modified database pages and current directory is discarded.
- The state of database before the failure occurs is now available through the shadow directory and this state can be recovered using shadow directory pages.
- This technique does not require any UNDO/REDO operation.

5.15 Check Points

- Checkpoint is a mechanism where all the previous logs are removed from the system and stored permanently in a storage disk.
- Checkpoint declares a point before which the DBMS was in consistent state, and all the transactions were committed.
- The recovery system reads the logs backwards from the end to the last checkpoint.
- Performing a checkpoint consists of the following operations :
  - Suspending executions of transactions temporarily;
  - Writing (force-writing) all modified database buffers of committed transactions out to disk;
  - Writing a checkpoint record to the log; and
  - Writing (force-writing) all log records in main memory out to disk.
- A checkpoint record usually contains additional information, including a list of transactions active at the time of the checkpoint.
- Many recovery methods (including the deferred and immediate update methods) need this information when a transaction is **rolled back**, as all transactions active at the time of the checkpoint and any subsequent ones may need to be redone.
- Since checkpoints cause some loss in performance while they are being taken, their frequency should be reduced if fast recovery is not critical.
- If we need fast recovery check-pointing frequency should be increased. If the amount of stable storage available is less, frequent check-pointing is unavoidable.

5.16 Log-based Recovery : Deferred and Immediate Update

SPPU : May-18,19, Marks 8

Before discussing the recovery algorithms(deferred and immediate update), let us see the concept of Log and REDO and UNDO operations.

5.16.1 Concept of Log

- Log is the most commonly used structure for recording the modifications that is to be made in the actual database. Hence during the recovery procedure a log file is maintained.
- A log record maintains four types of operations. Depending upon the type of operations there are four types of log records -
  1. **<Start>** Log record : It is represented as <T_i, Start>
  2. **<Update>** Log record
  3. **<Commit>** Log record : It is represented as <T_i, Commit>
  4. **<Abort>** Log record : It is represented as <T_i, Abort>
- The log contains various fields as shown in following Fig. 5.16.1. This structure is for <update> operation

Transaction ID(T_i)	Data Item Name	Old Value of Data Item	New Value of Data Item

Fig. 5.16.1

- For example : The sample log file is

<T_1, Start>
< T_1,a,10,20>
<T_1, Commit>

→ Here 10 represents the old value before commit operation and 20 is the new value that needs to be updated in the database after commit operation.

- The log must be maintained on the stable storage and the entries in the log file are maintained before actually updating the physical database.
- There are two approaches used for log based recovery technique - Deferred Database Modification and Immediate Database Modification.

5.16.2 REDO and UNDO Operation

During transaction execution, the updates are recorded only in the log and in the cache buffers. After the transaction reaches its commit point and the log is force written to disk and the updates are recorded in the database.

In order to maintain the atomicity of transaction, the operations can be redone or undone.

UNDO : This is an operation in which we restore all the old values (BFIM - BeFore Modification Image) onto the disk. This is called **roll-back operation**.

REDO : This is an operation in which all the modified values(AFIM - AFter Modification Image) are restored onto the disk. This is called **roll-forward operation**.

These operations are recorded in the log as they happen.

Difference between UNDO and REDO

Sr. No.	UNDO	REDO
1.	Makes a change go away.	Reproduces a change.
2.	Used for rollback and read consistency.	Used for rolling forward the changes.
3.	Protects the database from inconsistent reads.	Protects from data loss.

5.16.3 Write Ahead Logging Rule

- Before a block of data in main memory is output to the database, all log records pertaining to data in that block must have been output to stable storage. This rule is called the **Write-Ahead Logging (WAL)**
- This rule is necessary because - In the event of a crash or ROLLBACK, the original content contained in the rollback journal is played back into the database file to revert the database file to its original state.

5.16.4 Deferred Database Modification

- In this technique, the database is not updated immediately.
- Only log file is updated on each transaction.
- When the transaction reaches to its commit point, then only the database is physically updated from the log file.
- In this technique, if a transaction fails before reaching to its commit point, it will not have changed database anyway. Hence there is no need for the UNDO operation. The REDO operation is required to record the operations from log file to physical database. Hence deferred database modification technique is also called as **NO UNDO/REDO algorithm.**

- **For example :**

Consider two transactions T_1 and T_2 as follows :

T_1	T_2
Read (A, a)	Read (C, c)
a = a – 10	c = c – 20
Write (A, a)	Write (C, c)
Read (B, b)	
b = b + 10	
Write (B, b)	

If T_1 and T_2 are executed serially with initial values of A = 100, B = 200 and C = 300, then the state of log and database if crash occurs

a) Just after write (B, b)

b) Just after write (C, c)

c) Just after <T_2, commit>

The result of above 3 scenarios is as follows :

Initially the log and database will be

Log	Database
<T_1, Start>	
<T_1, A, 90>	
<T_1, B, 210>	
<T_1, Commit>	
	A = 90
	B = 210
<T_2, Start>	
<T_2, C, 280>	
<T_2, Commit>	
	C = 280

a) Just after write (B, b)

Just after write operation, no commit record appears in log. Hence no write operation is performed on database. So database retains only old values. Hence A = 100 and B = 200 respectively.

Thus the system comes back to original position and no redo operation take place.

The incomplete transaction of T_1 can be deleted from log.

b) Just after write (C, c)

The state of log records is as follows

Note that crash occurs before T_2 commits. At this point T_1 is completed successfully, so new values of A and B are written from log to database. But as T_2 is not committed, there is no redo (T_2) and the incomplete transaction T_2 can be deleted from log.

The redo (T_1) is done as < T_1, commit> gets executed. Therefore A = 90, B = 210 and

C = 300 are the values for database.

c) Just after < T_2, commit>

The log records are as follows :

< T_1, Start>

< T_1, A, 90>

< T_1, B, 210>

< T_1, Commit>

< T_2, Start>

< T_2, 6, 280>

< T_2, Commit>

← Crash occurs here

Clearly both T_1 and T_2 reached at commit point and then crash occurs. So both redo (T_1) and redo (T_2) are done and updated values will be A = 90, B = 210, C = 280.

5.16.5 Immediate Database Modification

In this technique, the database is updated during the execution of transaction even before it reaches to its commit point.

If the transaction gets failed before it reaches to its commit point, then the a **ROLLBACK Operation** needs to be done to bring the database to its earlier consistent state. That means the effect of operations need to be undone on the database. For that purpose both Redo and Undo operations are both required during the recovery. This technique is known as **UNDO/ REDO** technique.

For example : Consider two transaction T_1 and T_2 as follows :

T_1	T_2
Read(A,a)	Read(C, c)
a = a –10	c = c – 20
Write(A, a)	Write(C, c)
Read(B, b)	
b = b+10	
Write(B, b)	

Here T_1 and T_2 are executed serially. Initially A = 100, B = 200 and C = 300

If the crash occurs after

i) Just after Write(B, b) ii) Just after Write(C, c) iii) Just after <T_2,Commit>

Then using the immediate Database modification approach the result of above three scenarios can be elaborated as follows :

The contents of **log** and **database** is as follows :

Log	**Database**
<T_1,Start>	
<T_1,A,100,90>	
	A = 90
<T_1,B,200,210>	
	B = 210
<T_1,Commit>	
<T_2,Start>	
<T_2,C,300,280>	
	C = 280
<T_2,Commit>	

The recovery scheme uses two recovery techniques -

i) **UNDO (T_i) :** The transaction T_i needs to be undone if the log contains <T_i,Start> but does not contain <T_i,Commit>. In this phase, it restores the values of all data items updated by T_i to the old values.

ii) **REDO (T_i) :** The transaction T_i needs to be redone if the log contains both <T_i,Start> and <T_i,Commit>. In this phase, the data item values are set to the new values as per the transaction. After a failure has occurred log record is consulted to determine which transaction need to be redone.

a) Just after Write (B, b) : When system comes back from this crash, it sees that there is <T_1, Start> but no <T_1, Commit>. Hence T_1 must be undone. That means old values of A and B are restored. Thus old values of A and B are taken from log and both the transaction T_1 and T_2 are re-executed.

b) Just after Write (C, c) : Here both the redo and undo operations will occur.

c) Undo : When system comes back from this crash, it sees that there is <T_2, Start> but no <T_2, Commit>. Hence T_2 must be undone. That means old values of C is restored.

Thus old value of C is taken from log and the transaction T_2 is re-executed.

d) Redo : The transaction T_1 must be done as log contains both the <T_1,Start> and <T_1,Commit>

So A = 90, B = 210 and C = 300

e) Just after <T_2, Commit> : When the system comes back from this crash, it sees that there are two transaction T_1 and T_2 with both start and commit points. That means T_1 and T_2 need to be redone. So A = 90, B = 210 and C = 280

Example 5.16.1 *Suppose there is a database system that never fails. Is a recovery manager require for this system ? Why ?*

Solution :

(1) Yes. Even-though the database system never fails, the recovery manager is required for this system.

(2) During the transaction processing some transactions might be aborted. Such transactions must be rolled back and then the schedule is continued further.

(3) Thus to perform the rollbacks of aborted transactions recovery manager is required.

Review Question

1. *To ensure atomicity despite failures we uses recovery methods. Explain in detail log-based recovery method.* **SPPU : May-18,19 End Sem, Marks 8**

Unit - IV

Multiple Choice Questions

Q.1 The properties that are used in transaction management are _____.

a atomicity b consistency

c isolation d all of above

Q.2 Transaction must be _____.

a large b small

c atomic d all of the above

Q.3 Which of the following has "all-or-none" property ?

a Atomicity b Durability

c Isolation d All of the mentioned

Q.4 The initial state of transaction model is ___

a Init b Active

c Start d None of these

Q.5 When a transaction executes all its operations successfully , it enters into ____

a active b aborted

c partially committed d committed

Q.6 Transaction processing is associated with everything below except _____.

a producing detail, summery, or exception reports

b recording a business activity

c confirming an action or triggering a response

d maintaining data

Q.7 The property of transaction that persists all the crashes is _____.

a atomicity b durability

c isolation d all of the mentioned

Q.8 A set of changes that must be all made together is called as _____.

a concurrency b decomposition

c transaction d none of these

Q.9 Two actions on the same data object conflict if at least one of them is _____.

a read b write

c read-write d none of these

Q.10 The last step of transaction in which the transaction executes all its operations successfully is called as ___.

a active b aborted

c commit d partially commit

Q.11 The property in which all the transactions will be carried out and executed as if it is the only transaction in the system is known as _____.

a consistency b isolation

c durability d all of above

Q.12 When one schedule can be transformed to another schedule by series of swaps of non-conflicting instructions, then w say that two schedules are _____.

a equivalent b serial

c conflict Equivalent d none of these

Q.13 For a schedule S to be a conflict serializable following condition must hold true _____.

a there should be different transactions in the schedule.

b the operations must be performed on same data item.

c one of the operation must be write operations

d all of the above

Q.14 ____helps solve concurrency problem.

a Locking b Transaction monitor

c Transaction serializability d Two phase commit

Q.15 If a transaction acquires exclusive lock, then it can perform ____operations.

a read b write

c read and write d none of these

Q.16 If a transaction acquires a shared lock, then it can perform ____ operation.

a read b write

c read and write d none of these

Q.17 Which of the following disallows both dirty reads and nonrepeatable reads, but allows phantom reads ?

a Read committed b Read uncommitted

c Repeatable read d Serializable

Q.18 In a two-phase locking protocol, a transaction release locks in _____ phase.

a shrinking phase b growing phase

c running phase d initial phase

Q.19 No more lock request can be asked in ___ phase.

a shrinking phase b growing phase

c running phase d initial phase

Q.20 ____ is a specific concurrency problem wherein two transactions depend on each other for something.

a conflict b deadlock

c dirty read d uncommitted transaction

Answer Keys for Multiple Choice Questions :

Q.1	d	**Q.2**	c	**Q.3**	a	**Q.4**	b
Q.5	d	**Q.6**	c	**Q.7**	b	**Q.8**	c
Q.9	b	**Q.10**	c	**Q.11**	b	**Q.12**	c
Q.13	d	**Q.14**	a	**Q.15**	c	**Q.16**	a
Q.17	c	**Q.18**	a	**Q.19**	a	**Q.20**	b

❑❑❑

Notes